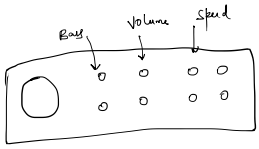
Designing & Debugging MLPs: From Architecture to Training

Thursday, 15 October 2026 5:00 PM

DJ mixer.



* Baseline model

→ Overfitting

→ High loss

→ Low Precision

→ Low Recall.

Q: How to improve NN performance?

1) Increase hidden layers. / Increase model complexity
2) Reduce hidden layers. / Reduce model complexity.
3) Increase number of epochs.
4) Regularization.
5) Learning Rate
6) Early stopping
7) Dropout
8) Batch Norm
9) Different Optimizers
10) Get more data.

All of these can be used to improve model performance,

↓

- It is important to know which technique to use when.

Scenario 1 : Model is Overfitting

Performs well on training data

Performs bad on test data.

1) Use simpler NN
2) Dropout → Quiz 1.
3) Use Regularization
4) Batch Normalization.
5) Diverse training data points.
6) Early Stopping:

Layer:

dropout_rate = 0.3 (turn off 30% of the neurons)

Scenario 2 : Model Underfits

Bad training accuracy.

1) More Epochs.
2) Deeper NN
3) Different Optimizer.
4) More data.

Rare.

Scenario 3 : Train and Valid Accuracy is good but Test is poor.

1) Check data leakage.
2) Bigger validation Set ✓

Q: What are the hyperparameters to look for while working on NN ?

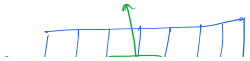
1) LR (η)
2) β in GD with momentum
3) β_1 & β_2 in Adam
4) No. of layers in NN
5) # of neurons / hidden units
6) LR Decay
7) Batch size

Q: What should be the order in which we tune the hyperparameters ?

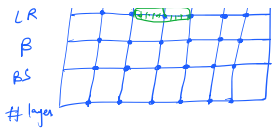
1) Learning Rate (η)
2) β value in GD with momentum
3) No. of layers / # of hidden units
4) Batch size
5) LR decay (LR reducing with # of epochs).
6) β_1 , β_2 of Adam. (Not Recommended to play around too much).

Q: How do we find out the value of the hyperparameter ?

- Grid Search. > Which one is optimal ? → Random Search. [. . .]
- Random Search.



Random Search → Grid Search.



Grid Search

1 value of BS → LR.
→ LR.

5 X 5 Grid search.

↑ ↑

BS LR.

Random Search → 25 LR. values. with 25 randomly selected Batch size.

LR ⟷ BS → ①

↑

(25)

0.01 ⟷ (128) → less important.

⋮

⋮ ⋯ ⟶ Best result.

25

H.W.

1) ✓ Gradient Checking : proves if our backpropogation is bug free

↓

Gradients.

↓

Small error → Our model's learning goes for a toss.

2) ✓ Macro & Micro F1, Precision & Recall.